

Docker Containerization

KAVIPRAKASH J

2023115104

Aim

To install Docker on Windows Subsystem for Linux (WSL), create and manage containers, and execute a Python environment inside a Docker container.

Objective

- Install Docker Engine in WSL.
 - Verify Docker functionality.
 - Run lightweight containers.
 - Deploy a web server container.
 - Execute Python inside a containerized environment.
-

System Requirements

- Windows 10/11 with WSL enabled
 - Ubuntu (WSL distribution)
 - Docker Engine
 - Internet connectivity
-

Procedure

Step 1 – Install Docker

Docker packages and dependencies were installed using the apt package manager.

Command used:

```
sudo apt update
```

```
sudo apt install docker.io
```

Step 2 – Start Docker Service

The Docker service was started and verified.

```
sudo systemctl start docker
```

```
docker --version
```

```
kavip@KaviPrakash:~$ sudo systemctl start docker
kavip@KaviPrakash:~$ docker --version
Docker version 28.2.2, build 28.2.2-0ubuntu1~24.04.1
kavip@KaviPrakash:~$
```

Step 3 – Test Docker with BusyBox

A simple container was executed to confirm Docker is working.

```
sudo docker run busybox echo "Docker is working successfully"
```

```
kavip@KaviPrakash: ~  
kavip@KaviPrakash:~$ docker run busybox echo "Docker is working successfully"  
Docker is working successfully
```

Step 4 – Run Alpine Linux Container

An interactive Alpine container was launched.

```
docker run -it alpine sh
```

This provided a minimal Linux environment inside Docker.

```
kavip@KaviPrakash:~$ docker run -it --name alpine_test alpine sh
Unable to find image 'alpine:latest' locally
latest: Pulling from library/alpine
589002ba0eae: Pull complete
Digest: sha256:25109184c71bdad752c8312a8623239686a9a2071e8825f20acb8f2198c3f659
Status: Downloaded newer image for alpine:latest
/ # uname -a
Linux 7e91307e5ac4 5.15.167.4-microsoft-standard-WSL2 #1 SMP Tue Nov 5 00:21:55 UTC 2024 x86_64 Linux
/ # ls /
bin    dev    etc    home   lib    media  mnt    opt    proc   root   run    sbin   srv    sys    tmp    usr    var
/ #
```

Step 5 – Deploy Apache Web Server

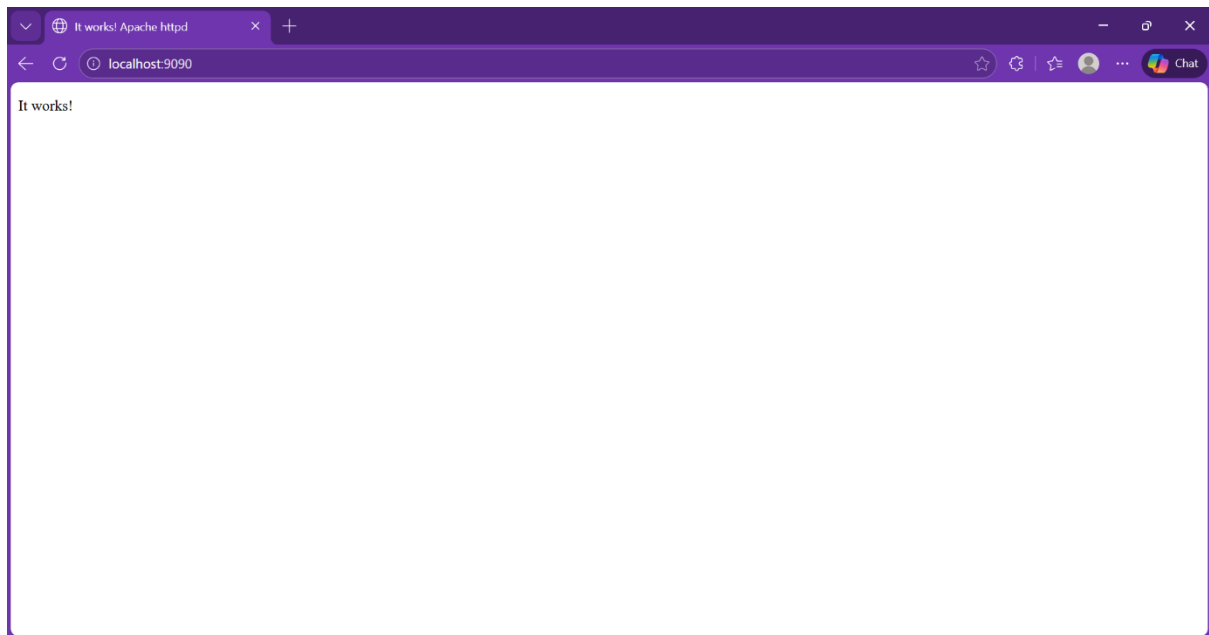
An Apache container was created and mapped to port 8080.

```
docker run -d -p 9090:80 httpd
```

```
docker ps
```

The Apache default webpage was successfully accessed through the browser.

```
kavip@KaviPrakash:~$ docker run -d --name apache_server -p 9090:80 httpd
Unable to find image 'httpd:latest' locally
latest: Pulling from library/httpd
0c8d55a45c0d: Pull complete
fe1db8dd446f: Pull complete
4f4fb700ef54: Pull complete
30973b009bab: Pull complete
85b0d820aa56: Pull complete
996228ca33ab: Pull complete
Digest: sha256:b89c19a390514d6767e8c62f29375d0577190be448f63b24f5f11d6b03f7bf18
Status: Downloaded newer image for httpd:latest
6a1b33988452b0f0e5ec0f143448d2c51a730335dfe7e1b171f951c727edd9a5
kavip@KaviPrakash:~$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
6a1b33988452   httpd     "httpd-foreground"      4 seconds ago Up 3 seconds  0.0.0.0:9090->80/tcp, [::]:9090->80/tcp  apache_server
kavip@KaviPrakash:~$
```



Step 6 – View Docker Containers

All containers were listed to verify their status.

`docker ps -a`

This displayed running and stopped containers.

```
kavip@KaviPrakash:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS
37f5c706a7fc   python:3.11-slim  "bash"                  2 minutes ago   Exited (2) About a minute ago
6a1b33988452   httpd          "httpd-foreground"      3 minutes ago   Up 3 minutes        0.0.0.0:9090->80/tcp, [::]
:9090->80/tcp
7e91307e5ac4   apache_server  "sh"                    6 minutes ago   Exited (130) 3 minutes ago
efdf2ac746b10  alpine_test    "echo 'Docker is wor..." 6 minutes ago   Exited (0) 6 minutes ago
boring_moser
f711f5735469  busybox        "echo 'Docker is wor..." 8 minutes ago   Exited (0) 8 minutes ago
nifty_margulis
kavip@KaviPrakash:~$
```

Step 7 – Run Python Container

A Python development container was launched.

```
docker run -it python:3.11-slim bash
```

```
python3
```

Python executed successfully inside the container.

```
kavip@KaviPrakash:~$ docker ps -a
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS              PORTS
37f5c706a7fc   python:3.11-slim  "bash"                  2 minutes ago  Exited (2) About a minute ago
6a1b33988452   py_dev         "httpd-foreground"     3 minutes ago  Up 3 minutes       0.0.0.0:9090->80/tcp, [::]
:9090->80/tcp   apache_server   "sh"                    6 minutes ago  Exited (130) 3 minutes ago
7e91307e5ac4   alpine_test     "echo 'Docker is wor..." 6 minutes ago  Exited (0) 6 minutes ago
efd2ac746b10   busybox        "echo 'Docker is wor..." 6 minutes ago  Exited (0) 6 minutes ago
f711f5735469   boring_moser    "echo 'Docker is wor..." 8 minutes ago  Exited (0) 8 minutes ago
nifty_margulis
kavip@KaviPrakash:~$ docker start -ai py_dev
root@37f5c706a7fc:/# python3
Python 3.11.14 (main, Feb  4 2026, 20:24:25) [GCC 14.2.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>> msg = "Welcome from Dockerized Python!"
g.upper()
>>> print(msg.upper())
WELCOME FROM DOCKERIZED PYTHON!
>>>
```

Output

- Docker Engine installed successfully.
- Containers were created and executed without errors.
- Apache web server ran successfully on localhost.
- Python environment functioned correctly inside Docker.

Result

Docker was successfully installed and configured in WSL. Multiple containers were created and managed, and a Python runtime environment was executed inside a Docker container, demonstrating effective containerization.